

Reto 4 – Seguridad y Control de Acceso con JWT

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. Hasta ahora, se ha construido un sistema distribuido con comunicación sincrónica y asincrónica. Sin embargo, todos los endpoints están "abiertos"; cualquier cliente que conozca la dirección IP y el puerto puede crear o eliminar empleados sin ninguna restricción.

En un entorno del mundo real, la seguridad es fundamental. Por ello, en este cuarto reto se incorporará autenticación y autorización utilizando el estándar **JSON Web Token (JWT)**.

Objetivo

Implementar un mecanismo de seguridad centralizado para el ecosistema de microservicios. Al finalizar este reto, el sistema debe ser capaz de autenticar a los usuarios a través de un nuevo servicio de identidad, emitir tokens JWT y validar dichos tokens en cada petición para proteger el acceso a los recursos existentes.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Implementar un nuevo microservicio **Servicio de Autenticación (auth-service)** encargado de verificar credenciales y generar el JWT.
- Proteger los endpoints de los servicios existentes (Empleados, Departamentos, Perfiles, Notificaciones) validando el JWT.
- El token JWT debe incluir **claims (reclamaciones)** específicas, como el rol de usuario, para implementar autorización.
- Implementar **Control de Acceso Basado en Roles (RBAC)**: ciertos endpoints solo deben ser accesibles para el rol `ADMIN` , mientras que otros pueden ser consumidos por el rol `USER` .
- Documentar cómo obtener el token y realizar peticiones autenticadas utilizando **OpenAPI (Swagger)**.

1. Servicio de Autenticación (auth-service)

Este nuevo servicio actuará como el proveedor de identidad del sistema.

Endpoints requeridos

Método	Ruta	Descripción
POST	/auth/login	Recibe credenciales (usuario y contraseña), verifica su validez y retorna un JWT.
POST	/auth/recover-password	Inicia el proceso de recuperación de cuenta recibiendo un email. Genera y publica un evento <code>usuario.recuperacion</code> .
POST	/auth/reset-password	Recibe el token de recuperación y la nueva contraseña para actualizarla en la base de datos.

Integración por Eventos (RabbitMQ / Kafka)

El **sistema de autenticación debe integrarse al ecosistema de eventos** (creado en el Reto 3) para automatizar el ciclo de vida de los usuarios:

Evento a Consumir	Acción en el auth-service
empleado.creado	Crea un usuario automáticamente sin contraseña válida (o inhabilitado). Genera un Token de Recuperación/Establecimiento de contraseña (ej. UUID) asociado a este usuario. Se le asigna el rol <code>USER</code> . Inmediatamente después, el servicio debe publicar un evento <code>usuario.creado</code> incluyendo el email y este token (nunca una contraseña en plano).
empleado.eliminado	Inhabilita o elimina lógicamente al usuario asociado, impidiendo futuros logins. y expira/elimina tokens de recuperación activos.

Eventos Salientes del auth-service

Evento a Publicar	Payload Sugerido	Disparador	Consumido por
usuario.creado	{ "email": "...", "token": "uuid-aqui" }	Consumo de empleado.creado	notificaciones-service
usuario.recuperacion	{ "email": "...", "token": "uuid-aqui" }	Petición a /auth/recover-password	notificaciones-service

Consumo en el notificaciones-service

El servicio de notificaciones construido en el Reto 3 debe actualizarse para escuchar los nuevos eventos emitidos por el `auth-service` . Al consumirlos, simulará el envío de un correo electrónico al empleado con un enlace o instrucciones para establecer su contraseña:

Para `usuario.creado` o `usuario.recuperacion` :

[NOTIFICACIÓN] Tipo: SEGURIDAD | Para: `juan@empresa.com` | Mensaje: "Para establecer o recuperar su contraseña, utilice e

(Nota: En un escenario de producción, este log simularía el envío de un correo con un link del estilo `https://app.empresa.com/reset?token=xyz123`)

Estructuras de Tokens (Acceso y Recuperación)

El sistema ahora manejará dos tipos de tokens con propósitos diferentes:

1. Token de Acceso (Access JWT)

Es el token que se retorna en `/auth/login` . Su payload debe tener, como mínimo, la siguiente estructura:

```
{
  "sub": "nombre_de_usuario",
  "role": "ADMIN | USER",
  "iat": 1712345678,
  "exp": 1712349278
}
```

Nota: La contraseña no debe incluirse **nunca** en el token JWT, y en la base de datos debe almacenarse encriptada/hasheada (ej. BCrypt).

2. Token de Recuperación/Establecimiento (Reset Token)

Es el token que se genera al crear un empleado o al solicitar recuperar la contraseña. Para su implementación, el equipo puede elegir una de las siguientes opciones:

Opción A (Recomendada - Stateless con JWT): Reusar la librería JWT para generar un token firmado específicamente para este propósito. Debe incluir un *claim* especial para distinguirlo de un token de acceso y un tiempo de expiración corto (ej. 15 minutos a 1 hora).

```
{
  "sub": "nombre_de_usuario",
  "type": "RESET_PASSWORD",
  "iat": 1712345678,
  "exp": 1712349278
}
```

Ventaja: El `auth-service` puede validar matemáticamente su autenticidad y expiración sin necesidad de consultar ni guardar el token en la base de datos.

Opción B (Stateful con UUID): Generar un UUID aleatorio (ej. `d3b07384-d9a7...`) y guardarlo en la base de datos asociado al usuario, junto con una fecha de expiración (`expiresAt`).

Ventaja: Es fácil de revocar inmediatamente si el usuario lo usa.

Desventaja: Requiere tablas adicionales en base de datos o almacenamiento en caché (Redis).

2. Protección de los Microservicios (Validación JWT)

Una vez que el `auth-service` puede emitir tokens, los demás servicios deben exigir este token en la cabecera HTTP `Authorization` usando el esquema `Bearer`.

Formato esperado en las peticiones

Los clientes deberán enviar el token en todas las peticiones a los recursos protegidos:

```
Authorization: Bearer <token_jwt_aqui>
```

Opciones de Implementación para la Validación (Elegir una)

- API Gateway (Recomendado):** Introducir un Gateway (ej. Spring Cloud Gateway, Nginx, o Express Gateway) que reciba todas las peticiones externas, valide el JWT interceptando la solicitud, y luego la enrute al microservicio correspondiente.
- Middleware/Interceptor por Servicio:** Configurar cada microservicio individual (Empleados, Departamentos, etc.) con una librería de seguridad que intercepte y valide la firma del JWT antes de procesar el controlador.

Reglas de Autorización (RBAC)

Debe aplicar las siguientes reglas utilizando la información del token:

- Rol ADMIN :** Tiene acceso total. Puede crear, modificar y eliminar recursos en cualquier servicio. (ej. `DELETE /empleados/{id}`, `POST /departamentos`).
- Rol USER :** Tiene acceso de solo lectura. Solo puede consultar información. (ej. `GET /empleados`, `GET /perfiles/{id}`).
- Peticiones sin Token válido:** Deben recibir un código HTTP `401 Unauthorized`.
- Peticiones de un rol sin permisos:** Deben recibir un código HTTP `403 Forbidden`.

3. Pruebas del Sistema

Verificar el funcionamiento completo del flujo de seguridad:

Flujo de prueba sugerido

1. Iniciar todos los servicios: `docker-compose up --build`
2. **Onboarding de empleado:** Petición `POST /empleados` (abierto por defecto sin token o simular una creación directa si el endpoint ya está protegido para que el Admin inicial lo cree).
Nota: Generalmente se requiere un usuario Administrador "semilla" (seed) creado por base de datos o variable de entorno al levantar el servicio.
3. **Verificar Eventos:** Confirmar en los logs del `auth-service` que se detectó `empleado.creado` y se generó el usuario. Luego, confirmar en los logs de `notificaciones-service` que se detectó `usuario.creado` y se simuló el envío de credenciales.
4. **Petición denegada:** Intentar acceder a `GET /empleados` sin cabeceras. (Debe fallar con `401 Unauthorized`).
5. **Establecer Contraseña (Reset):** Extraer el token de recuperación de los logs de la notificación. Enviar una petición `POST /auth/reset-password` con el token y la nueva contraseña deseada.
6. **Login USER :** Realizar login en `/auth/login` utilizando el correo del empleado recién creado y la contraseña que acaba de establecer. Extraer el token JWT de la respuesta.
7. **Lectura exitosa:** Hacer una petición a `GET /empleados` enviando el JWT en la cabecera `Authorization: Bearer <token>`. (Debe retornar `200 OK`).
8. **Prueba de Recuperación:** Enviar `POST /auth/recover-password` con el email del usuario. Verificar en los logs de notificaciones que se generó un nuevo evento `usuario.recuperacion` con un nuevo token. Repetir el paso 5 con este nuevo token y el paso 6 para validar el cambio.
9. **Escritura denegada (403):** Intentar hacer `DELETE /empleados/E001` con el JWT del usuario regular. (Debe fallar con `403 Forbidden`).
10. **Offboarding (Admin):** Utilizando el JWT del administrador "semilla", enviar el comando `DELETE /empleados/E001`. (Debe retornar `200 OK` o `204 No Content` y publicar `empleado.eliminado`).
11. **Verificar Inhabilitación:** Intentar hacer login nuevamente con las credenciales del empleado eliminado. (Debe fallar con `401 Unauthorized` o `403 Forbidden`).

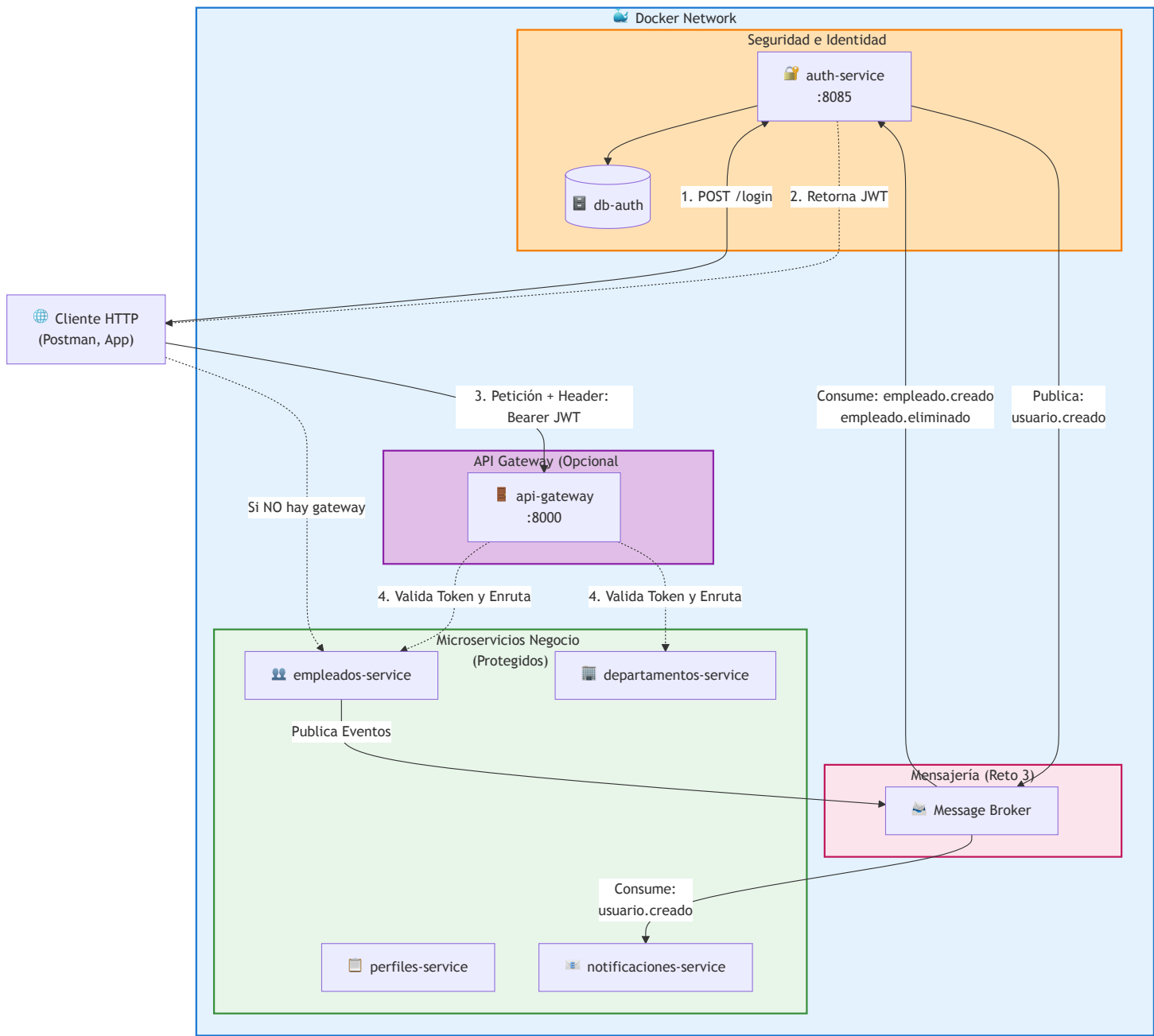
Entregables

- Código versionado en sus respectivos repositorios de GitHub.
- Configuración de `docker-compose.yml` actualizada que incluya el nuevo `auth-service` y, si se usa, el componente API Gateway.
- Modificar el [README.md](#) para incluir:
 - Instrucciones de cómo obtener un token para realizar pruebas locales (ej. colección de Postman/Bruno).
 - Explicación de qué estrategia de validación de token se seleccionó (Gateway vs. Interceptores individuales) y por qué.
 - La clave secreta (secret key) del JWT (únicamente por propósitos académicos) documentada o definida claramente en el `.env.example`.

Consideraciones

- Tanto el `auth-service` (que firma el token) como los servicios que lo validan deben usar el **mismo "Secret"** o clave simétrica. En un entorno de microservicios real con clave asimétrica, compartirían la clave pública, pero por simplicidad académica, una firma simétrica (HMAC SHA-256) es suficiente.
- El secreto debe inyectarse a todos los contenedores a través de **variables de entorno** en Docker Compose.
- Asegúrese de que la UI de Swagger esté configurada con el esquema de seguridad `BearerAuth` para permitir probar las APIs cómodamente sin usar 100% herramientas como Postman.

Diagrama de Arquitectura Esperada



Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, correspondiendo **1 punto** a cada uno de los elementos principales:

#	Elemento	Valor	Aspectos a evaluar
1	Servicio de Autenticación	1.0	Creación del auth-service , almacenamiento seguro de credenciales (hash), emisión correcta de tokens con formato estándar JWT.
2	Validación de Token	1.0	Los servicios exigen el JWT y validan su firma correctamente. Se rechazan peticiones sin token o con token alterado (401).

#	Elemento	Valor	Aspectos a evaluar
3	Control de Acceso (RBAC)	1.0	Se respetan los roles ADMIN y USER . Las acciones no autorizadas son correctamente frenadas devolviendo un código 403 .
4	Gestión de variables de entorno	1.0	El "Secret" del JWT y otras configuraciones se inyectan correctamente vía docker-compose.yml a los contenedores necesarios, y no están "quemadas" (hardcoded) en el código.
5	Documentación e Integración	1.0	Sistema completo levantable con docker-compose up , OpenAPI (Swagger) soportando el esquema BearerAuth, e instrucciones claras en el README.